

MODUL AJAR

Penyimpanan Data pada Web

Cookies, Session, dan Local Storage

Dilengkapi contoh kode, tampilan layar (screenshot) untuk setiap program, kuis, dan rubrik penilaian.

Sasaran: mahasiswa yang sudah menguasai CRUD PHP. **Estimasi waktu:** 3 pertemuan.

Cara memakai modul ini

1. Baca materi dan tujuan pada setiap tahap.
2. Ketik dan jalankan kode di server lokal (XAMPP atau `php -S localhost:8000`).
3. Bandingkan hasilnya dengan bagian TAMPILAN LAYAR di bawah tiap program.
4. Kerjakan aktivitas, tugas mandiri, dan kuis.

Catatan: tampilan DevTools dan Console pada gambar adalah ilustrasi yang menampilkan data asli hasil menjalankan kode. Pada saat praktik, buka DevTools dengan tombol F12 (tab Application untuk Cookies dan Local Storage). Screenshot dibuat dengan PHP 8.3; database memakai SQLite untuk demonstrasi, dan tampilannya identik dengan MySQL.

Daftar Isi

Jadwal Pertemuan	3
Tahap 1: Pengantar	4
Tahap 2: Cookies (PHP)	7
Tahap 3: Session (PHP)	13
Tahap 4: Local Storage (JavaScript)	17
Tahap 5: Perbandingan dan Keamanan	24
Tahap 6: Latihan dan Evaluasi	Error! Bookmark not defined.
Bank Soal Kuis	26
Lampiran: Ringkasan Cepat (Cheat Sheet)	Error! Bookmark not defined.

Jadwal Pertemuan

Modul ajar untuk mahasiswa yang sudah menguasai CRUD PHP. Estimasi waktu: 3 pertemuan.

Pertemuan	Cakupan
1	Tahap 1 (Pengantar) dan Tahap 2 (Cookies)
2	Tahap 3 (Session) dan Tahap 4 (Local Storage)
3	Tahap 5 (Perbandingan dan Keamanan) dan Tahap 6 (Latihan dan Evaluasi)

Tahap 1: Pengantar

Tujuan

Mahasiswa memahami mengapa web membutuhkan mekanisme penyimpanan data dan mampu menyebutkan tiga mekanismenya.

Materi

- HTTP bersifat *stateless*: server tidak mengingat pengguna dari satu request ke request berikutnya.
- Kebutuhan nyata: status login, keranjang belanja, dan preferensi tampilan.
- Tiga mekanisme:
 - **Cookies**: disimpan di browser, dikirim ke server pada setiap request.
 - **Session**: data disimpan di server, browser hanya menyimpan ID sesi.
 - **Local Storage**: disimpan di browser, dibaca lewat JavaScript, tidak dikirim ke server.

Demo: melihat data di DevTools

1. Buka situs apa saja, tekan F12.
2. Pilih tab Application (Chrome/Edge) atau Storage (Firefox).
3. Buka bagian Cookies dan Local Storage, lalu lihat isinya.

Demo kode: membuktikan HTTP stateless

Demo ini memakai tiga halaman. Pada request ke-2 server **benar-benar menerima** sebuah data (nama). Pada request ke-3, yang hanya satu klik setelahnya, server sudah tidak mengingatnya. Tidak ada variabel yang kita reset sendiri: server memang tidak mengaitkan dua request sebagai pengguna yang sama.

langkah1.php (Request 1: form input nama)

```
<!DOCTYPE html>
<html lang="id">
<head><meta charset="UTF-8"><title>Langkah 1</title>
<style>body{font-family:sans-serif;padding:1.5rem}
.tag{background:#1a57a0;color:#fff;padding:2px 10px;border-radius:10px;font-size:13px}</style></head>
<body>
  <span class="tag">Request 1</span>
  <h2>Langkah 1: Perkenalkan diri</h2>
  <form method="post" action="langkah2.php">
    <input type="text" name="nama" placeholder="Nama Anda" required>
    <button type="submit">Kirim ke server</button>
  </form>
</body>
</html>
```

langkah2.php (Request 2: server menerima data)

```
<?php
// Request 2: server MENERIMA nama dari form (data hanya ada di request ini)
$nama = $_POST['nama'] ?? null;
?>
<!DOCTYPE html>
<html lang="id">
<head><meta charset="UTF-8"><title>Langkah 2</title>
```

```
<style>body{font-family:sans-serif;padding:1.5rem}
.tag{background:#1a7f37;color:#fff;padding:2px 10px;border-radius:10px;font-
size:13px}</style></head>
<body>
  <span class="tag">Request 2</span>
  <h2>Langkah 2: Server menerima data</h2>
  <p>Halo, <b><?= htmlspecialchars($nama ?? 'tidak diketahui') ?></b>! Server berhasil
menerima nama Anda.</p>
  <a href="langkah3.php">Lanjut ke Langkah 3 &raquo;</a>
</body>
</html>
```

langkah3.php (Request 3: request baru, data tidak terbawa)

```
<?php
// Request 3: request BARU. Tidak ada data dari request 2 yang terbawa.
$nama = $_POST['nama'] ?? null;
?>
<!DOCTYPE html>
<html lang="id">
<head><meta charset="UTF-8"><title>Langkah 3</title>
<style>body{font-family:sans-serif;padding:1.5rem}
.tag{background:#c0392b;color:#fff;padding:2px 10px;border-radius:10px;font-
size:13px}</style></head>
<body>
  <span class="tag">Request 3</span>
  <h2>Langkah 3: Apakah server masih ingat?</h2>
  <p>Halo, <b><?= htmlspecialchars($nama ?? 'tidak diketahui') ?></b>.</p>
  <p>Server <b>lupa</b> nama Anda, padahal baru saja menerimanya di request sebelumnya.</p>
</body>
</html>
```

TAMPILAN LAYAR (hasil menjalankan kode di atas)



Gambar 1. *langkah1.php*: pengguna mengisi nama "Budi" lalu menekan tombol kirim (Request 1).



Gambar 2. *langkah2.php*: server menerima dan menampilkan nama "Budi" dari `$_POST` (Request 2).



Gambar 3. *langkah3.php*: setelah mengklik tautan, server tidak tahu lagi siapa pengguna dan menampilkan "tidak diketahui" (Request 3). Inilah arti HTTP stateless.

Pertanyaan diskusi: Bagaimana caranya agar *langkah3.php* tetap bisa menampilkan "Halo, Budi"? (Jawaban: nama harus disimpan di suatu tempat dan dibawa pada request berikutnya, yaitu dengan cookies, session, atau penyimpanan lain yang dibahas di tahap berikutnya. Contoh: tambahkan `$_SESSION['nama'] = $nama;` di *langkah2.php* dan baca kembali di *langkah3.php*.)

Tahap 2: Cookies (PHP)

Tujuan

Mahasiswa mampu membuat, membaca, dan menghapus cookie serta menjelaskan batasannya.

Materi

- `setcookie(nama, nilai, kedaluwarsa, path, domain, secure, httponly)`
- `$_COOKIE['nama']` untuk membaca.
- Menghapus cookie: atur masa berlaku ke masa lalu.
- `setcookie()` harus dipanggil sebelum ada output HTML apa pun.
- Cookie baru terbaca di `$_COOKIE` pada request berikutnya, bukan pada request yang sama.

Contoh 1: mengingat nama pengguna dan menghitung kunjungan

`cookie_dasar.php`

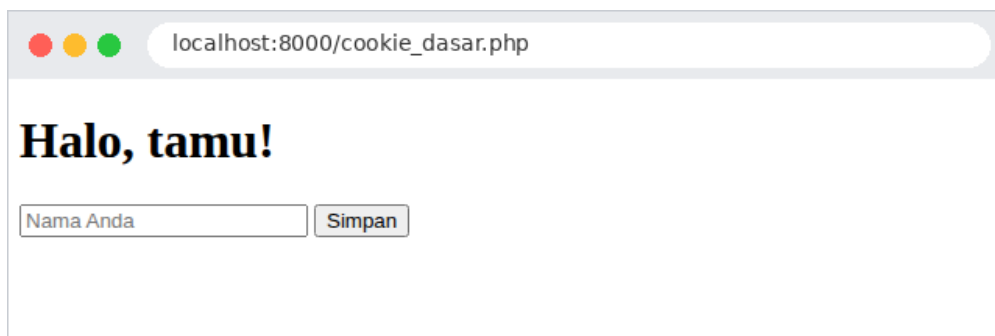
```
<?php
// Simpan nama jika form dikirim
if (isset($_POST['nama']) && trim($_POST['nama']) != '') {
    setcookie('nama', trim($_POST['nama']), time() + (86400 * 7), '/');
    header('Location: cookie_dasar.php');
    exit;
}

// Hapus nama
if (isset($_GET['hapus'])) {
    setcookie('nama', '', time() - 3600, '/');
    header('Location: cookie_dasar.php');
    exit;
}

// Hitung kunjungan (diletakkan SETELAH semua exit, agar request
// yang hanya berupa redirect tidak ikut dihitung)
$kunjungan = isset($_COOKIE['kunjungan']) ? (int)$_COOKIE['kunjungan'] : 0;
$kunjungan++;
setcookie('kunjungan', $kunjungan, time() + (86400 * 7), '/'); // 7 hari

$nama = $_COOKIE['nama'] ?? null;
?>
<!DOCTYPE html>
<html lang="id">
<head><meta charset="UTF-8"><title>Cookie Dasar</title></head>
<body>
    <?php if ($nama): ?>
        <h1>Halo, <?= htmlspecialchars($nama) ?>!</h1>
        <p>Anda sudah membuka halaman ini <?= $kunjungan ?> kali.</p>
        <a href="?hapus=1">Lupakan saya</a>
    <?php else: ?>
        <h1>Halo, tamu!</h1>
        <form method="post">
            <input type="text" name="nama" placeholder="Nama Anda" required>
            <button type="submit">Simpan</button>
        </form>
    <?php endif; ?>
</body>
</html>
```

TAMPILAN LAYAR (hasil menjalankan kode di atas)



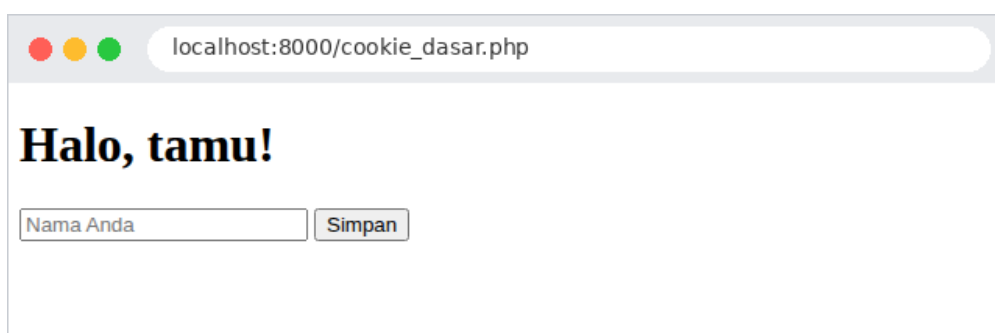
Gambar 4. Kunjungan pertama ke `cookie_dasar.php`: belum ada cookie nama, sehingga form input tampil.



Gambar 5. Setelah nama "Budi" disimpan dan halaman dialihkan (redirect): halaman menyapa dari isi cookie. Angka kunjungan menjadi 2, bertambah 1 dari kunjungan pertama.

Elements	Console	Network	Application	Ilustrasi DevTools (F12)		
STORAGE		Name	Value	Path	Expires	HttpOnly
▶ Local Storage localhost:8000		nama	Budi	/	2026-10-11 08:41	
▶ Session Storage localhost:8000		kunjungan	2	/	2026-10-11 08:41	
▶ Cookies localhost:8000						

Gambar 6. Isi cookie nama dan kunjungan di DevTools, tab Application > Cookies (klik panah di samping Cookies, lalu pilih alamat situs). Perhatikan kolom Expires (7 hari) dan Path.



Gambar 7. Setelah menekan "Lupakan saya": cookie nama dihapus dan form tampil lagi.

Mengapa penghitung diletakkan di bawah blok exit?

Setiap request ke halaman ini menjalankan skrip dari atas. Saat form dikirim atau tautan "Lupakan saya" diklik, skrip berakhir di `header(...)` dan `exit`, lalu browser membuat **request baru** akibat redirect. Jika baris penghitung diletakkan **di atas** blok tersebut, kedua request dihitung sehingga

angkanya naik 2. Jika diletakkan **di bawah** semua **exit**, hanya request yang benar-benar menampilkan halaman yang dihitung.

Aksi pengguna	Penghitung di atas blok exit	Penghitung di bawah blok exit
Buka halaman pertama kali	1	1
Refresh (F5)	+1	+1
Klik Simpan (POST lalu redirect)	+2	+1
Klik Lupakan saya (GET lalu redirect)	+2	+1

Pelajaran: urutan kode di PHP menentukan hasilnya. Kode yang berada sebelum **exit** tetap dijalankan pada request yang berujung redirect, sedangkan kode setelahnya tidak.

Aktivitas

1. Jalankan **cookie_dasar.php**, isi nama, lalu amati angka kunjungan.
2. Buka DevTools > Application > Cookies, klik panah di samping **Cookies**, lalu pilih alamat situs untuk melihat cookie **nama** dan **kunjungan**.
3. Pindahkan blok "Hitung kunjungan" ke bagian paling atas skrip, lalu klik Simpan dan Lupakan saya. Apa yang berubah pada angka kunjungan? Jelaskan penyebabnya.
4. Buka tab Network (centang Preserve log), lalu klik Simpan. Perhatikan dua request **cookie_dasar.php**: yang pertama berstatus 302 (redirect), yang kedua 200.

Contoh 2: keranjang belanja dengan cookies

keranjang_cookie.php

Array keranjang diubah menjadi string JSON karena cookie hanya menyimpan teks.

```
<?php
$produk = [
    1 => ['nama' => 'Buku Tulis', 'harga' => 5000],
    2 => ['nama' => 'Pulpen',      'harga' => 3000],
    3 => ['nama' => 'Penggaris',   'harga' => 4000],
];

// Baca keranjang dari cookie
$keranjang = isset($_COOKIE['keranjang']) ? json_decode($_COOKIE['keranjang'], true) : [];
if (!is_array($keranjang)) {
    $keranjang = [];
}

// Tambah produk
if (isset($_GET['tambah'])) {
    $id = (int)$_GET['tambah'];
    if (isset($produk[$id])) {
        $keranjang[$id] = ($keranjang[$id] ?? 0) + 1;
        setcookie('keranjang', json_encode($keranjang), time() + 3600, '/');
    }
    header('Location: keranjang_cookie.php');
    exit;
}

// Kosongkan keranjang
if (isset($_GET['kosongkan'])) {
    setcookie('keranjang', '', time() - 3600, '/');
    header('Location: keranjang_cookie.php');
    exit;
}
```

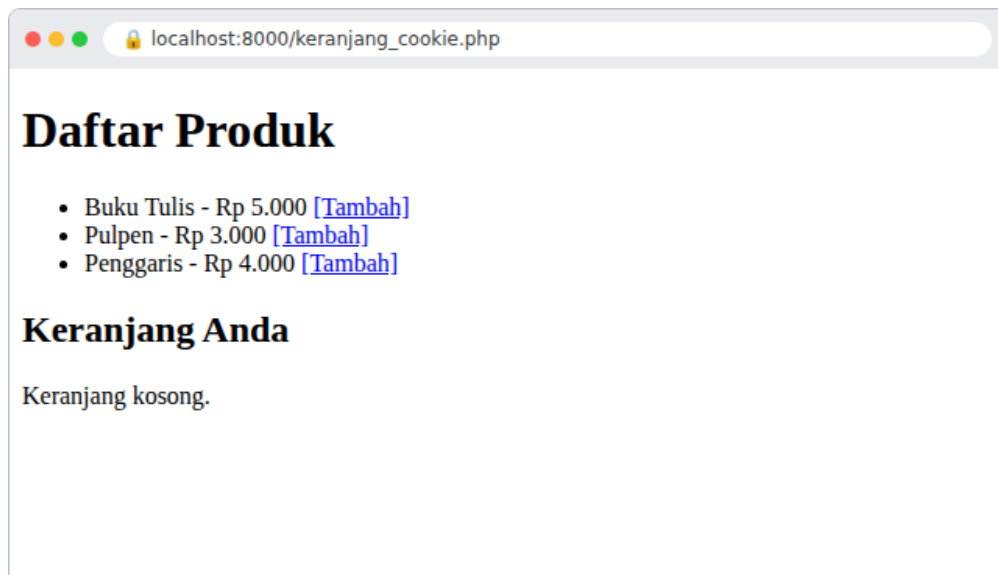
```

?>
<!DOCTYPE html>
<html lang="id">
<head><meta charset="UTF-8"><title>Keranjang (Cookies)</title></head>
<body>
    <h1>Daftar Produk</h1>
    <ul>
    <?php foreach ($produk as $id => $p): ?>
        <li>
            <? = htmlspecialchars($p['nama']) ?> - Rp <? = number_format($p['harga'], 0, ',',
            '('.') ?>
            <a href="?tambah=?= $id ?>">[Tambah]</a>
        </li>
    <?php endforeach; ?>
    </ul>

    <h2>Keranjang Anda</h2>
    <?php if (empty($keranjang)): ?>
        <p>Keranjang kosong.</p>
    <?php else: ?>
        <?php $total = 0; ?>
        <ul>
        <?php foreach ($keranjang as $id => $jumlah): ?>
            <?php if (!isset($produk[$id])) continue; ?>
            <?php $sub = $produk[$id]['harga'] * (int)$jumlah; $total += $sub; ?>
            <li><? = htmlspecialchars($produk[$id]['nama']) ?> x <? = (int)$jumlah ?>
                = Rp <? = number_format($sub, 0, ',', '.') ?></li>
        <?php endforeach; ?>
        </ul>
        <p><strong>Total: Rp <? = number_format($total, 0, ',', '.') ?></strong></p>
        <a href="?kosongkan=1">Kosongkan keranjang</a>
    <?php endif; ?>
</body>
</html>

```

TAMPILAN LAYAR (hasil menjalankan kode di atas)



Gambar 8. Halaman *keranjang_cookie.php* saat cookie keranjang belum ada: keranjang kosong.



Gambar 9. Setelah menekan [Tambah] pada Buku Tulis (2 kali) dan Pulpen (1 kali), keranjang dan total harga tampil.

Elements	Console	Network	Application	Ilustrasi DevTools (F12)		
STORAGE		Name	Value	Path	Expires	HttpOnly
▶ Local Storage localhost:8000		keranjang	%7B%221%22%3A2%2C%222%22%3A1%7D	/	2026-09-28 04:13	
▶ Session Storage localhost:8000						
▶ Cookies localhost:8000						

Gambar 10. Cookie keranjang berisi teks JSON: kunci = id produk, nilai = jumlah. Data ini tersimpan di browser.

Elements	Console	Network	Application	Ilustrasi DevTools (F12)		
STORAGE		Name	Value	Path	Expires	HttpOnly
▶ Local Storage localhost:8000		keranjang	{"1":999,"2":1}	/	2026-09-28 04:13	
▶ Session Storage localhost:8000						
▶ Cookies localhost:8000						

Gambar 11. **Percobaan keamanan:** nilai cookie diubah manual di DevTools (jumlah Buku Tulis menjadi 999).



Gambar 12. Setelah refresh, server memercayai nilai cookie sehingga total berubah menjadi Rp 4.998.000. Ini bukti cookie mudah dimanipulasi pengguna.

Aktivitas

1. Jalankan `keranjang_cookie.php`, tambahkan beberapa produk.
2. Buka DevTools, lihat cookie `keranjang`.
3. **Ubah nilainya secara manual** (misalnya jumlah menjadi 999), lalu refresh. Diskusikan mengapa ini menjadi masalah keamanan.

Batasan cookies

- Kapasitas sekitar 4 KB per cookie.
- Ikut terkirim di setiap request, sehingga memperbesar lalu lintas data.
- Mudah dilihat dan diubah pengguna.

Diskusi :

Untuk keranjang belanja akan lebih baik pakai session, mengapa demikian ?

Tahap 3: Session (PHP)

Tujuan

Mahasiswa mampu membuat sistem login dan logout dengan session serta melindungi halaman.

Materi

- `session_start()` harus dipanggil di awal setiap halaman yang memakai session.
- Data disimpan di server; browser hanya menyimpan ID sesi (cookie `PHPSESSID`).
- `$_SESSION['kunci'] = nilai` untuk menyimpan, `unset($_SESSION['kunci'])` untuk menghapus satu data, `session_destroy()` untuk mengakhiri sesi.
- `session_regenerate_id(true)` setelah login berhasil, untuk mencegah *session fixation*.

Persiapan database

```
CREATE DATABASE db_belajar;
USE db_belajar;

CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    nama_lengkap VARCHAR(100) NOT NULL
);
```

Buat data awal dengan skrip `seed_user.php` (jalankan sekali, lalu hapus filenya):

```
<?php
$pdo = new PDO('mysql:host=localhost;dbname=db_belajar;charset=utf8mb4', 'root', '');
$hash = password_hash('rahasia123', PASSWORD_DEFAULT);
$stmt = $pdo->prepare('INSERT INTO users (username, password, nama_lengkap) VALUES (?, ?, ?)');
$stmt->execute(['budi', $hash, 'Budi Santoso']);
echo 'User dibuat.';
```

`koneksi.php`

```
<?php
$pdo = new PDO(
    'mysql:host=localhost;dbname=db_belajar;charset=utf8mb4',
    'root',
    '',
    [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
);
```

`login.php`

```
<?php
session_start();
require 'koneksi.php';

// Jika sudah login, langsung ke dashboard
if (isset($_SESSION['user_id'])) {
    header('Location: dashboard.php');
    exit;
}

$error = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $username = trim($_POST['username'] ?? '');
    $password = $_POST['password'] ?? '';
```

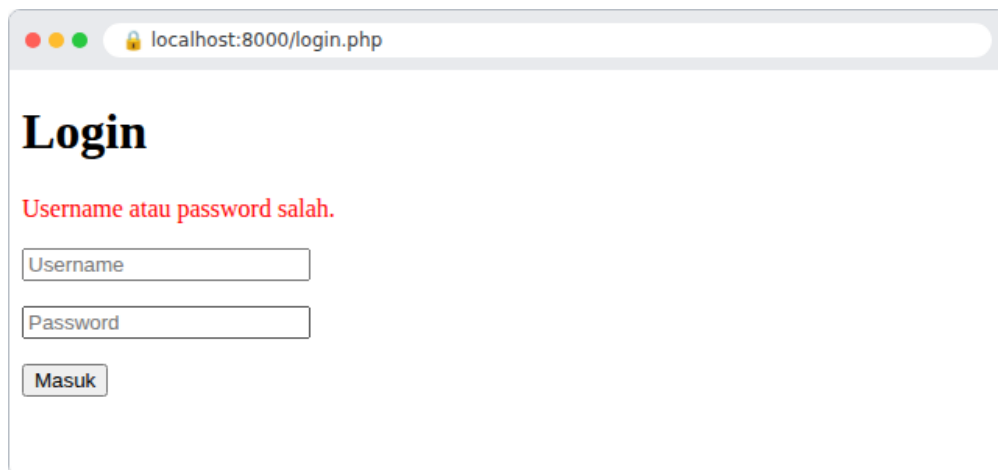
```

$stmt = $pdo->prepare('SELECT * FROM users WHERE username = ?');
$stmt->execute([$username]);
$user = $stmt->fetch(PDO::FETCH_ASSOC);

if ($user && password_verify($password, $user['password'])) {
    session_regenerate_id(true); // cegah session fixation
    $_SESSION['user_id'] = $user['id'];
    $_SESSION['nama'] = $user['nama_lengkap'];
    header('Location: dashboard.php');
    exit;
}
$error = 'Username atau password salah.';
}
?>
<!DOCTYPE html>
<html lang="id">
<head><meta charset="UTF-8"><title>Login</title></head>
<body>
    <h1>Login</h1>
    <?php if ($error): ?>
        <p style="color:red"><?= htmlspecialchars($error) ?></p>
    <?php endif; ?>
    <form method="post">
        <p><input type="text" name="username" placeholder="Username" required></p>
        <p><input type="password" name="password" placeholder="Password" required></p>
        <button type="submit">Masuk</button>
    </form>
</body>
</html>

```

TAMPILAN LAYAR (hasil menjalankan kode di atas)



Gambar 13. Login dengan password salah: pesan kesalahan tampil dan pengguna tetap di **login.php**.

dashboard.php (halaman terproteksi)

```

<?php
session_start();

if (!isset($_SESSION['user_id'])) {
    header('Location: login.php');
    exit;
}
?>
<!DOCTYPE html>
<html lang="id">
<head><meta charset="UTF-8"><title>Dashboard</title></head>
<body>
    <h1>Selamat datang, <?= htmlspecialchars($_SESSION['nama']) ?>!</h1>

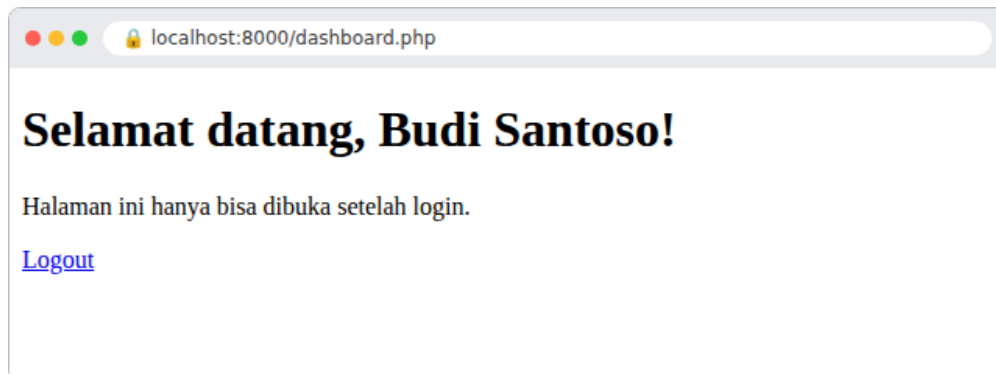
```

```

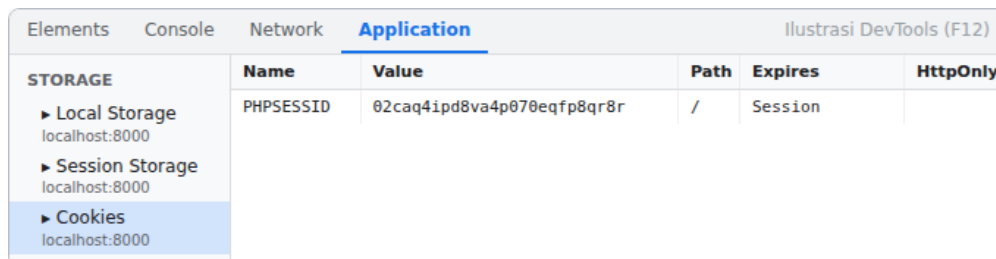
<p>Halaman ini hanya bisa dibuka setelah login.</p>
<a href="logout.php">Logout</a>
</body>
</html>

```

TAMPILAN LAYAR (hasil menjalankan kode di atas)



Gambar 14. Login berhasil (user budi, password rahasia123): pengguna dialihkan ke **dashboard.php** dan nama diambil dari `$_SESSION`.



Gambar 15. Di browser hanya ada cookie **PHPSESSID** berisi ID acak. Nama dan data pengguna **tidak** disimpan di browser, melainkan di server.



Gambar 16. Membuka **dashboard.php** tanpa login (misalnya di jendela incognito): pengguna otomatis dialihkan ke **login.php**.

logout.php

```

<?php
session_start();

$_SESSION = [];           // kosongkan data sesi

if (ini_get('session.use_cookies')) {
    $p = session_get_cookie_params();

```

```

    setcookie(session_name(), '', time() - 42000, $p['path'], $p['domain'], $p['secure'],
    $p['httponly']);
}

session_destroy();           // hancurkan sesi di server
header('Location: login.php');
exit;

```

TAMPILAN LAYAR (hasil menjalankan kode di atas)



Gambar 17. Setelah menekan Logout: sesi dihancurkan dan pengguna kembali ke halaman login.

Elements	Console	Network	Application	Ilustrasi DevTools (F12)		
STORAGE		Name	Value	Path	Expires	HttpOnly
- Local Storage localhost:8000 - Session Storage localhost:8000 - Cookies localhost:8000		PHPSESSID	7ni2lak99ianpdb4bi7pf60jgn	/	Session	

Gambar 18. Cookie **PHPSESSID** sudah dihapus dari browser setelah logout (tabel kosong).

Aktivitas

1. Login, lalu lihat cookie **PHPSESSID** di DevTools. Perhatikan bahwa yang ada hanya ID acak, bukan nama atau data pengguna.
2. Coba buka **dashboard.php** di jendela *incognito* tanpa login. Apa yang terjadi?
3. Hapus cookie **PHPSESSID** secara manual, lalu refresh **dashboard.php**.

Diskusi

Mengapa status login disimpan di session dan bukan di cookie biasa? (Jawaban: cookie biasa bisa diubah pengguna, sedangkan data session ada di server sehingga tidak bisa dimanipulasi langsung.)

Tahap 4: Local Storage (JavaScript)

Tujuan

Mahasiswa mampu menyimpan dan membaca data di Local Storage, termasuk objek dan array.

Materi

- Kapasitas sekitar 5 MB per origin, tidak kadaluwarsa otomatis, tidak dikirim ke server.
- Metode: `setItem(key, value)`, `getItem(key)`, `removeItem(key)`, `clear()`, dan properti `length`.
- Semua nilai berupa **string**; objek dan array harus memakai `JSON.stringify()` dan `JSON.parse()`.
- `getItem` mengembalikan `null` jika kunci tidak ada.
- `sessionStorage` memiliki API yang sama, tetapi datanya hilang saat tab ditutup.

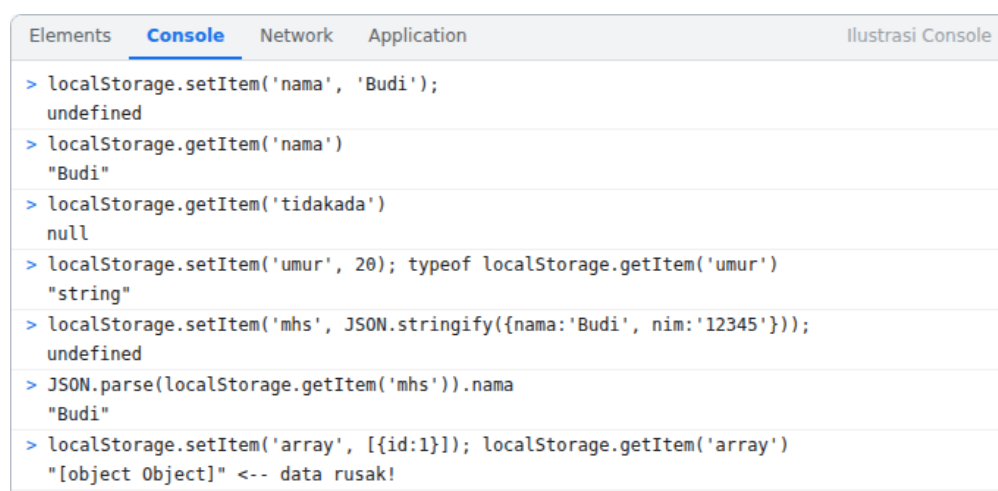
Contoh 0: metode dasar di Console

```
localStorage.setItem('nama', 'Budi');
console.log(localStorage.getItem('nama')); // "Budi"
console.log(localStorage.getItem('tidakada')); // null
localStorage.removeItem('nama');
localStorage.clear();

// Jebakan tipe data
localStorage.setItem('umur', 20);
console.log(typeof localStorage.getItem('umur')); // "string"

// Objek harus diubah ke JSON
const mhs = { nama: 'Budi', nim: '12345' };
localStorage.setItem('mhs', JSON.stringify(mhs));
const hasil = JSON.parse(localStorage.getItem('mhs'));
console.log(hasil.nama); // "Budi"
```

TAMPILAN LAYAR (hasil menjalankan kode di atas)



Gambar 19. Mencoba metode Local Storage langsung di Console DevTools. Perhatikan tipe string pada angka, dan data array yang rusak menjadi "[object Object]" jika tidak memakai `JSON.stringify()`.

Contoh 1: tema gelap/terang

tema.html

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Tema dengan Local Storage</title>
  <style>
    body { font-family: sans-serif; padding: 2rem; }
    body.gelap { background: #1e1e1e; color: #f0f0f0; }
  </style>
</head>
<body>
  <h1>Pengaturan Tema</h1>
  <p>Pilihan tema Anda tetap tersimpan setelah halaman di-refresh atau browser ditutup.</p>
  <button id="btnTema">Ganti Tema</button>

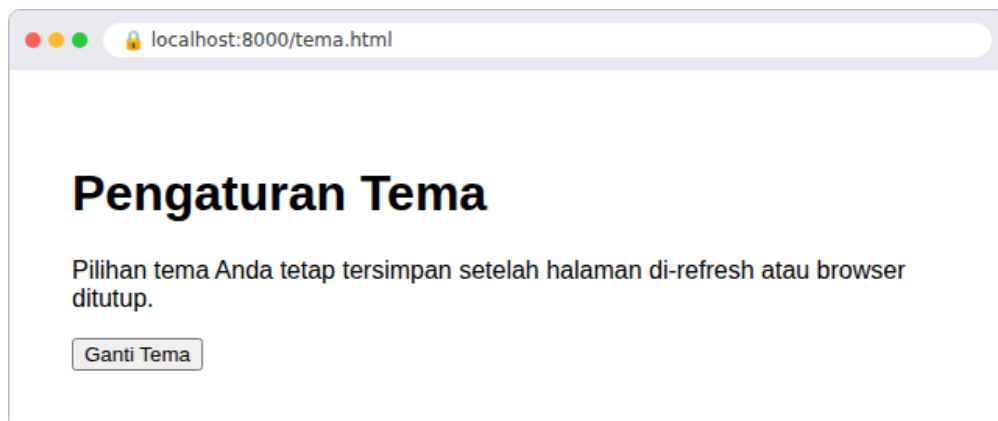
  <script>
    const KUNCI = 'tema';

    function terapkanTema(tema) {
      document.body.classList.toggle('gelap', tema === 'gelap');
    }

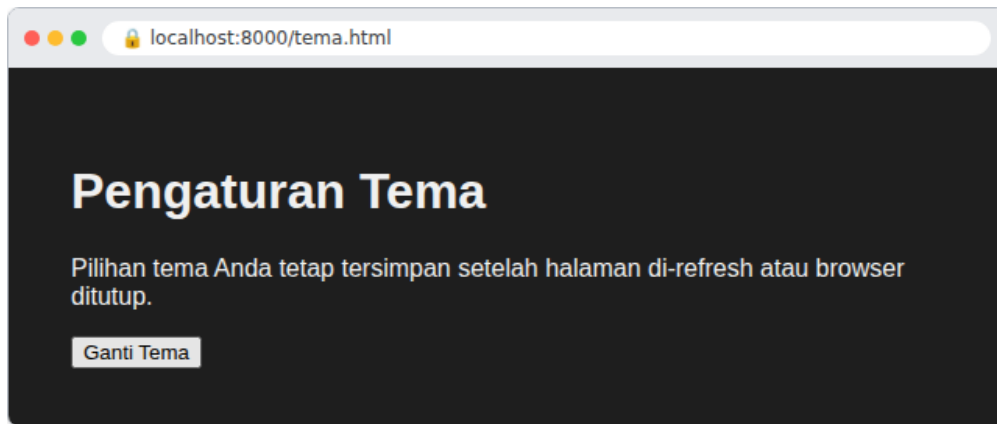
    // Saat halaman dibuka, baca tema tersimpan
    terapkanTema(localStorage.getItem(KUNCI) || 'terang');

    document.getElementById('btnTema').addEventListener('click', function () {
      const baru = document.body.classList.contains('gelap') ? 'terang' : 'gelap';
      localStorage.setItem(KUNCI, baru);
      terapkanTema(baru);
    });
  </script>
</body>
</html>
```

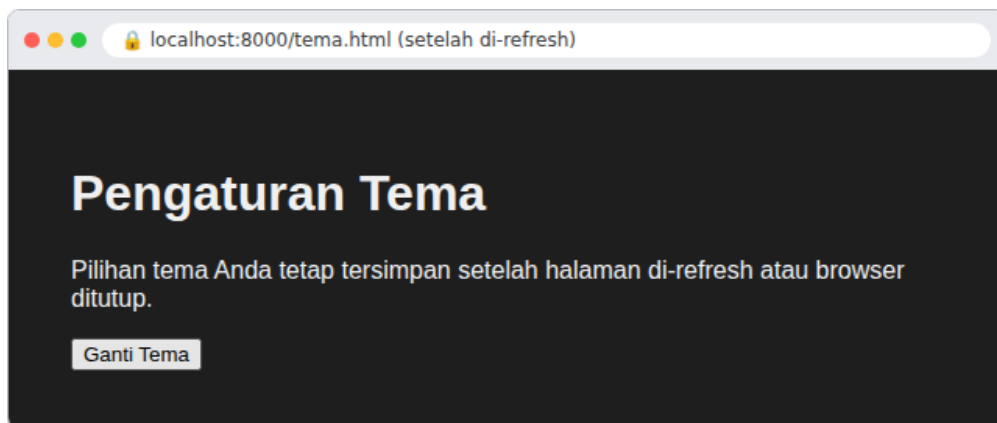
TAMPILAN LAYAR (hasil menjalankan kode di atas)



Gambar 20. *tema.html* pada kunjungan pertama: tema terang (nilai bawaan).



Gambar 21. Setelah menekan tombol Ganti Tema: tampilan menjadi gelap.



Gambar 22. Setelah halaman di-refresh: tema gelap tetap dipakai karena dibaca kembali dari Local Storage.

Elements	Console	Network	Application	Ilustrasi DevTools (F12)
STORAGE				
- Local Storage localhost:8000 - Session Storage localhost:8000 - Cookies localhost:8000		Key	Value	
		tema	gelap	

Gambar 23. Isi Local Storage di DevTools: satu pasangan kunci-nilai tema = gelap.

Contoh 2: keranjang belanja dengan Local Storage

keranjang_ls.html

Kasusnya sama dengan Tahap 2, agar mahasiswa bisa membandingkan langsung.

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Keranjang (Local Storage)</title>
</head>
<body>
  <h1>Daftar Produk</h1>
  <ul id="daftarProduk"></ul>

  <h2>Keranjang Anda</h2>
  <ul id="isiKeranjang"></ul>
  <p><strong id="total"></strong></p>
  <button id="btnKosongkan">Kosongkan keranjang</button>
```

```

<script>
  const produk = [
    { id: 1, nama: 'Buku Tulis', harga: 5000 },
    { id: 2, nama: 'Pulpen', harga: 3000 },
    { id: 3, nama: 'Penggaris', harga: 4000 }
  ];
  const KUNCI = 'keranjang';

  function bacaKeranjang() {
    try {
      return JSON.parse(localStorage.getItem(KUNCI)) || [];
    } catch (e) {
      return [];
    }
  }

  function simpanKeranjang(data) {
    localStorage.setItem(KUNCI, JSON.stringify(data));
  }

  function rupiah(angka) {
    return 'Rp ' + angka.toLocaleString('id-ID');
  }

  function tambah(id) {
    const keranjang = bacaKeranjang();
    const item = keranjang.find(function (i) { return i.id === id; });
    if (item) {
      item.jumlah++;
    } else {
      keranjang.push({ id: id, jumlah: 1 });
    }
    simpanKeranjang(keranjang);
    tampilkanKeranjang();
  }

  function tampilkanProduk() {
    const ul = document.getElementById('daftarProduk');
    produk.forEach(function (p) {
      const li = document.createElement('li');
      li.textContent = p.nama + ' - ' + rupiah(p.harga) + ' ';
      const btn = document.createElement('button');
      btn.textContent = 'Tambah';
      btn.addEventListener('click', function () { tambah(p.id); });
      li.appendChild(btn);
      ul.appendChild(li);
    });
  }

  function tampilkanKeranjang() {
    const ul = document.getElementById('isiKeranjang');
    ul.innerHTML = '';
    let total = 0;
    const keranjang = bacaKeranjang();
    if (keranjang.length === 0) {
      ul.innerHTML = '<li>Keranjang kosong.</li>';
    }
    keranjang.forEach(function (item) {
      const p = produk.find(function (x) { return x.id === item.id; });
      if (!p) return;
      const sub = p.harga * item.jumlah;
      total += sub;
      const li = document.createElement('li');
      li.textContent = p.nama + ' x ' + item.jumlah + ' = ' + rupiah(sub);
    });
  }

```

```

        ul.appendChild(li);
    });
    document.getElementById('total').textContent = 'Total: ' + rupiah(total);
}

document.getElementById('btnKosongkan').addEventListener('click', function () {
    localStorage.removeItem(KUNCI);
    tampilkanKeranjang();
});

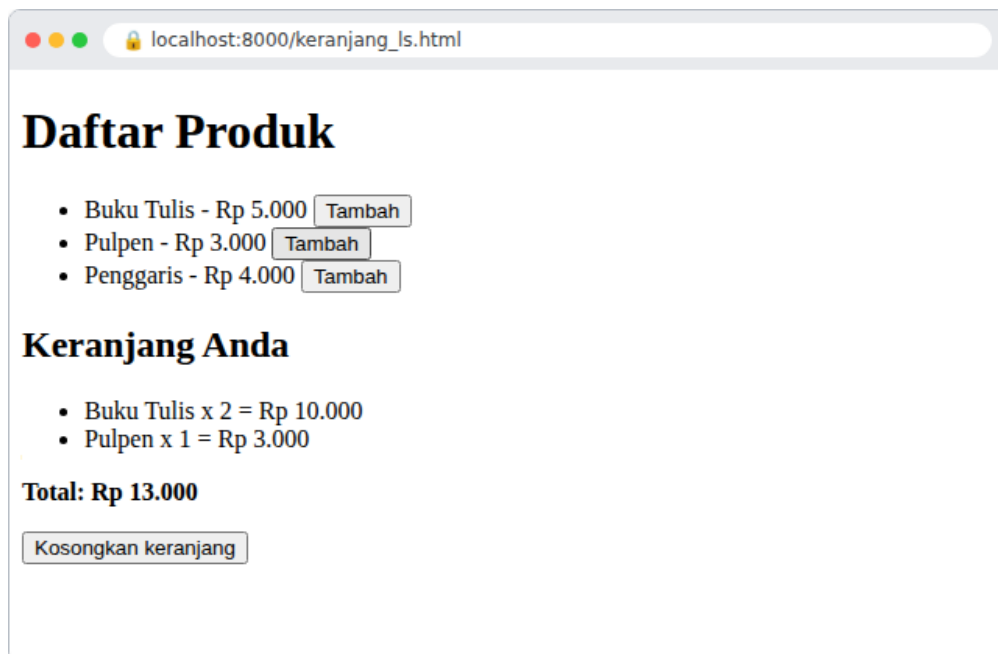
tampilkanProduk();
tampilkanKeranjang();
</script>
</body>
</html>

```

TAMPILAN LAYAR (hasil menjalankan kode di atas)



Gambar 24. *keranjang_ls.html* saat Local Storage belum berisi keranjang.



Gambar 25. Setelah menekan Tambah pada Buku Tulis (2 kali) dan Pulpen (1 kali). Hasilnya sama dengan versi cookies pada Tahap 2.

Elements	Console	Network	Application	Ilustrasi DevTools (F12)
STORAGE		Key	Value	
▶ Local Storage localhost:8000		keranjang	[{"id":1,"jumlah":2},{"id":2,"jumlah":1}]	
▶ Session Storage localhost:8000				
▶ Cookies localhost:8000				

Gambar 26. Di Local Storage, kunci keranjang berisi array JSON. Bedakan dengan versi cookies: data ini **tidak** dikirim ke server.

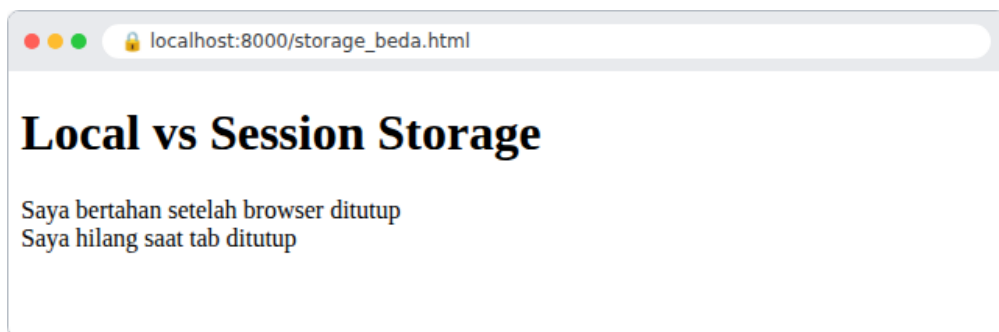
Contoh 3: perbedaan localStorage dan sessionStorage

storage_beda.html

```
<!DOCTYPE html>
<html lang="id">
<head><meta charset="UTF-8"><title>Local vs Session Storage</title></head>
<body>
  <h1>Local vs Session Storage</h1>
  <p id="hasil"></p>
  <script>
    localStorage.setItem('dari_local', 'Saya bertahan setelah browser ditutup');
    sessionStorage.setItem('dari_session', 'Saya hilang saat tab ditutup');

    document.getElementById('hasil').innerHTML =
      localStorage.getItem('dari_local') + '<br>' +
      sessionStorage.getItem('dari_session');
  </script>
</body>
</html>
```

TAMPILAN LAYAR (hasil menjalankan kode di atas)



Gambar 27. Halaman `storage_beda.html` menampilkan data dari kedua jenis storage.

Elements	Console	Network	Application	Ilustrasi DevTools (F12)
STORAGE		Local Storage		
▶ Local Storage localhost:8000		Key	Value	
		dari_local	Saya bertahan setelah browser ditutup	
▶ Session Storage localhost:8000		tema	gelap	
▶ Cookies localhost:8000		keranjang	[{"id":1,"jumlah":2},{"id":2,"jumlah":1}]	
		Session Storage		
		Key	Value	
		dari session	Saya hilang saat tab ditutup	

Gambar 28. Di DevTools, `dari_local` ada di Local Storage (permanen), sedangkan `dari_session` ada di Session Storage (hilang saat tab ditutup).

Aktivitas: buka file, tutup tab, buka lagi di tab baru, lalu bandingkan isi kedua storage di DevTools.

Bandingkan dengan Tahap 2

Ubah nilai keranjang di DevTools (Application > Local Storage). Apakah ini lebih aman dibanding cookie? (Jawaban: tidak; keduanya bisa diubah pengguna. Bedanya, Local Storage tidak ikut terkirim ke server.)

Tahap 5: Perbandingan dan Keamanan

Tabel perbandingan

Aspek	Cookies	Session (PHP)	Local Storage	sessionStorage
Lokasi data	Browser	Server (ID di browser)	Browser	Browser
Kapasitas	~4 KB	Bergantung server	~5 MB	~5 MB
Masa berlaku	Diatur lewat expires	Sampai sesi berakhir/timeout	Permanen sampai dihapus	Sampai tab ditutup
Dikirim ke server	Ya, setiap request	Hanya ID sesi	Tidak	Tidak
Diakses oleh	PHP dan JavaScript (kecuali HttpOnly)	PHP	JavaScript	JavaScript
Cocok untuk	Preferensi kecil yang dibaca server	Status login, data sensitif	Data besar atau preferensi sisi klien	Data sementara per tab

Panduan memilih

- **Status login dan data sensitif:** session.
- **Preferensi kecil yang perlu dibaca server** (misalnya bahasa): cookies.
- **Data besar atau tampilan sisi klien** (tema, draft, keranjang tanpa login): Local Storage.
- **Keranjang pengguna yang sudah login:** sebaiknya di database agar konsisten di semua perangkat.

Cookie yang lebih aman

```
setcookie('bahasa', 'id', [  
    'expires' => time() + 86400 * 30,  
    'path'    => '/',  
    'secure'  => true,           // hanya lewat HTTPS  
    'httponly' => true,         // tidak bisa dibaca JavaScript  
    'samesite' => 'Lax'        // mengurangi risiko CSRF  
]);
```

Demo risiko XSS pada Local Storage

Jika sebuah halaman rentan XSS, skrip penyerang dapat membaca seluruh isi Local Storage:

```
// Skrip berbahaya yang berhasil disisipkan ke halaman  
console.log(JSON.stringify(localStorage));
```

TAMPILAN LAYAR (hasil menjalankan kode di atas)



Gambar 29. Skrip apa pun yang berjalan di halaman dapat membaca **seluruh** isi Local Storage. Inilah alasan data sensitif tidak boleh disimpan di sini.

Karena itu:

- Jangan simpan password, token akses, atau data pribadi sensitif di Local Storage maupun cookie biasa.
- Selalu escape output: `htmlspecialchars()` di PHP, `textContent` (bukan `innerHTML`) di JavaScript untuk data dari pengguna.
- Gunakan flag `HttpOnly` untuk cookie yang tidak perlu dibaca JavaScript.

Tambahan pengaturan session yang aman

```
// Letakkan sebelum session_start()
session_set_cookie_params([
    'lifetime' => 0,
    'path'     => '/',
    'secure'   => true,
    'httponly' => true,
    'samesite' => 'Lax'
]);
session_start();
```

Soal dan Jawaban (Untuk Pemahaman)

A. Pilihan ganda

1. Protokol HTTP disebut *stateless* karena...

- A. Data tidak pernah dikirim ke server
- B. Server tidak mengingat request sebelumnya secara otomatis
- C. Server tidak dapat menyimpan data
- D. Browser tidak dapat menyimpan data

Jawaban: B. HTTP memperlakukan setiap request secara independen.

2. Fungsi PHP untuk membuat cookie adalah...

- A. `set_cookie()`
- B. `cookie_start()`
- C. `setcookie()`
- D. `$_COOKIE[]`

Jawaban: C. `$_COOKIE` hanya untuk membaca.

3. Cara menghapus cookie di PHP adalah...

- A. `deletecookie('nama')`
- B. Mengatur masa berlaku cookie ke waktu yang sudah lewat
- C. `unset($_COOKIE['nama'])` saja sudah cukup
- D. `session_destroy()`

Jawaban: B. `unset($_COOKIE[...])` hanya menghapus di variabel PHP pada request itu, bukan di browser.

4. Fungsi yang wajib dipanggil sebelum memakai `$_SESSION` adalah...

- A. `session_create()`
- B. `session_start()`
- C. `start_session()`
- D. `session_open()`

Jawaban: B.

5. Pada session PHP, yang disimpan di browser adalah...

- A. Seluruh isi `$_SESSION`
- B. Password pengguna
- C. ID sesi (`PHPSESSID`)
- D. Tidak ada yang disimpan

Jawaban: C. Data sesi sebenarnya ada di server.

6. Kapasitas Local Storage per origin kira-kira...

- A. 4 KB
- B. 5 MB
- C. 500 MB
- D. Tidak terbatas

Jawaban: B.

7. Hasil dari `typeof localStorage.getItem('umur')` setelah `localStorage.setItem('umur', 20)` adalah...

- A. number
- B. integer
- C. string
- D. object

Jawaban: C. Local Storage hanya menyimpan string.

8. Agar array JavaScript dapat disimpan di Local Storage, harus dilakukan...

- A. `localStorage.setItem('a', array)`
- B. `localStorage.setItem('a', JSON.stringify(array))`
- C. `localStorage.array = array`
- D. `localStorage.push(array)`

Jawaban: B.

9. Perbedaan utama sessionStorage dan localStorage adalah...

- A. sessionStorage dapat dibaca PHP
- B. sessionStorage memiliki kapasitas tak terbatas
- C. Data sessionStorage hilang saat tab ditutup
- D. sessionStorage dikirim ke server

Jawaban: C.

10. Flag cookie yang mencegah cookie dibaca oleh JavaScript adalah...

- A. Secure
- B. HttpOnly
- C. SameSite
- D. Path

Jawaban: B.

11. Setelah login berhasil, `session_regenerate_id(true)` dipanggil untuk...

- A. Mempercepat login
- B. Menghapus semua cookie
- C. Mencegah *session fixation*
- D. Menyimpan password

Jawaban: C.

12. Data berikut yang TIDAK boleh disimpan di Local Storage adalah...

- A. Pilihan tema
- B. Draft catatan
- C. Token akses atau password
- D. Bahasa tampilan

Jawaban: C. Local Storage dapat dibaca oleh skrip apa pun di halaman yang sama (risiko XSS).

B. Benar atau salah

No	Pernyataan	Jawaban
13	Cookie ikut terkirim ke server pada setiap request ke domain yang sama.	Benar

No	Pernyataan	Jawaban
14	Data Local Storage otomatis dikirim ke server bersama request HTTP.	Salah, tidak dikirim otomatis
15	<code>setcookie()</code> boleh dipanggil setelah ada output HTML.	Salah, harus sebelum output (header sudah terkirim)
16	Cookie yang baru dibuat langsung tersedia di <code>\$_COOKIE</code> pada request yang sama.	Salah, baru tersedia pada request berikutnya
17	Data yang disimpan di session PHP berada di sisi server.	Benar
18	Keranjang belanja di cookie tidak bisa diubah pengguna karena tersimpan di browser.	Salah, pengguna bisa mengubahnya lewat DevTools

C. Isian singkat

19. Tuliskan perintah JavaScript untuk menghapus data berkunci `tema` dari Local Storage.

Jawaban: `localStorage.removeItem('tema');`

20. Tuliskan perintah PHP untuk menyimpan id user ke session dengan kunci `user_id`.

Jawaban: `$_SESSION['user_id'] = $id; (setelah session_start())`

21. Perintah PHP untuk mengakhiri sesi adalah

Jawaban: `session_destroy();`

D. Studi kasus: pilih mekanisme dan jelaskan alasannya

No	Kasus	Jawaban yang diharapkan
22	Status login pengguna pada portal kampus	Session: data harus aman di server dan tidak boleh diubah pengguna
23	Pilihan bahasa tampilan yang perlu dibaca server saat membuat halaman	Cookie: kecil dan perlu terkirim ke server
24	Draft artikel yang panjang, disimpan otomatis saat mengetik	Local Storage: data besar, tidak perlu ke server
25	Nomor langkah pada formulir pendaftaran bertahap dalam satu tab	sessionStorage (atau session PHP jika perlu divalidasi server)
26	Keranjang belanja pelanggan yang login dan ingin melihatnya dari HP maupun laptop	Database (dikaitkan dengan akun): data harus konsisten antar perangkat